



University of Chittagong

Department of Computer Science & Engineering

Lab Report

Assembly Language Programming

CSE 416

Microprocessors and Embedded Systems Lab

Submitted By:
Debashish Chakraborty
ID: 23701034

Submitted To:
Dr. Atiqur Rahman
Associate Professor

November 5, 2025

Contents

1	Hello World	3
1.1	Problem Statement	3
1.2	Assembly Code	3
1.3	Output	3
2	Two Number Addition	4
2.1	Problem Statement	4
2.2	Assembly Code	4
2.3	Input	7
2.4	Output	7
3	Average Calculator	7
3.1	Problem Statement	7
3.2	Assembly Code	7
3.3	Input	11
3.4	Output	11
4	Number Comparison	11
4.1	Problem Statement	11
4.2	Assembly Code	11
4.3	Input	13
4.4	Output	14
5	Number Swapping	14
5.1	Problem Statement	14
5.2	Assembly Code	14
5.3	Input	17
5.4	Output	17
6	Prime Number Checker	18
6.1	Problem Statement	18
6.2	Assembly Code	18
6.3	Input	20
6.4	Output	20
7	Fibonacci Series Generator	21
7.1	Problem Statement	21
7.2	Assembly Code	21
7.3	Input	24
7.4	Output	24
8	Factorial Calculator	25
8.1	Problem Statement	25
8.2	Assembly Code	25
8.3	Input	28
8.4	Output	28
9	GCD Calculator	28

9.1	Problem Statement	28
9.2	Assembly Code	28
9.3	Input	31
9.4	Output	31
10	LCM Calculator	32
10.1	Problem Statement	32
10.2	Assembly Code	32
10.3	Input	35
10.4	Output	35
11	Left Triangle Pattern	35
11.1	Problem Statement	35
11.2	Assembly Code	35
11.3	Input	37
11.4	Output	37
12	Star Pyramid Pattern	38
12.1	Problem Statement	38
12.2	Assembly Code	38
12.3	Input	40
12.4	Output	40
13	Matrix Addition	41
13.1	Problem Statement	41
13.2	Assembly Code	41
13.3	Input	47
13.4	Output	47
14	Matrix Subtraction	48
14.1	Problem Statement	48
14.2	Assembly Code	48
14.3	Input	54
14.4	Output	54

1 Hello World

1.1 Problem Statement

Write an 8086 assembly program to display "Hello World" on the screen.

1.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    MSG DB 'Hello World$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, MSG
    MOV AH, 09H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

1.3 Output

Output

Hello World

2 Two Number Addition

2.1 Problem Statement

Prompt the user to enter two integers, calculate their sum, and display the result.

2.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT1 DB 'Enter first number: $'
    PROMPT2 DB 'Enter second number: $'
    RESULT_MSG DB 0DH, 0AH, 'The sum is: $'
    NUM1 DW ?
    NUM2 DW ?
    SUM DW ?
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for first number
    LEA DX, PROMPT1
    MOV AH, 09H
    INT 21H

    ; Read first number
    CALL READ_NUMBER
    MOV NUM1, AX

    ; Print newline
    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    ; Prompt for second number
    LEA DX, PROMPT2
    MOV AH, 09H
    INT 21H

    ; Read second number
    CALL READ_NUMBER
    MOV NUM2, AX

    ; Calculate sum
    MOV AX, NUM1
```

```
    ADD AX, NUM2
    MOV SUM, AX

    ; Display result message
    LEA DX, RESULT_MSG
    MOV AH, 09H
    INT 21H

    ; Display sum
    MOV AX, SUM
    CALL PRINT_NUMBER

    ; Print newline
    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    ; Exit
    MOV AH, 4CH
    INT 21H
MAIN ENDP

READ_NUMBER PROC
    PUSH BX
    PUSH CX
    PUSH DX

    XOR BX, BX

READ_LOOP:
    MOV AH, 01H
    INT 21H

    CMP AL, 0DH
    JE READ_DONE

    CMP AL, '0'
    JB READ_LOOP
    CMP AL, '9'
    JA READ_LOOP

    SUB AL, '0'
    MOV CL, AL
    MOV AX, BX
    MOV DX, 10
    MUL DX
    ADD AX, CX
    MOV BX, AX
    JMP READ_LOOP

READ_DONE:
```

```
    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_NUMBER ENDP

PRINT_NUMBER PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    XOR CX, CX
    MOV BX, 10

    CMP AX, 0
    JNE DIVIDE_LOOP
    PUSH AX
    INC CX
    JMP PRINT_DIGITS

DIVIDE_LOOP:
    XOR DX, DX
    DIV BX
    PUSH DX
    INC CX
    CMP AX, 0
    JNE DIVIDE_LOOP

PRINT_DIGITS:
    POP DX
    ADD DL, '0'
    MOV AH, 02H
    INT 21H
    LOOP PRINT_DIGITS

    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_NUMBER ENDP

END MAIN
```

2.3 Input

Input

```
Enter first number: 10
Enter second number: 20
```

2.4 Output

Output

```
The sum is: 30
```

3 Average Calculator

3.1 Problem Statement

Prompt for N numbers, compute average and print with two decimal places.

3.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT_COUNT DB 'How many numbers do you want to enter? $
    ,
    PROMPT_NUM DB 0DH, 0AH, 'Enter number $'
    COLON_MSG DB ': $'
    RESULT_MSG DB 0DH, 0AH, 'The average is: $'
    DOT_MSG DB '.$'
    COUNT DW ?
    SUM DW 0
    AVERAGE DW ?
    REMAINDER DW ?
    DECIMAL1 DB ?
    DECIMAL2 DB ?
    NUM_INDEX DW 1
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
```



```
; Prompt for count
LEA DX, PROMPT_COUNT
MOV AH, 09H
INT 21H

CALL READ_NUMBER
MOV COUNT, AX

MOV SUM, 0
MOV NUM_INDEX, 1

INPUT_LOOP:
; Display prompt
LEA DX, PROMPT_NUM
MOV AH, 09H
INT 21H

MOV AX, NUM_INDEX
CALL PRINT_NUMBER

LEA DX, COLON_MSG
MOV AH, 09H
INT 21H

CALL READ_NUMBER
ADD SUM, AX

INC NUM_INDEX
MOV AX, NUM_INDEX
CMP AX, COUNT
JBE INPUT_LOOP

; Calculate average
MOV AX, SUM
XOR DX, DX
DIV COUNT
MOV AVERAGE, AX
MOV REMAINDER, DX

; Calculate decimal places
MOV AX, REMAINDER
MOV BX, 100
MUL BX
DIV COUNT

XOR DX, DX
MOV BX, 10
DIV BX
MOV DECIMAL1, AL
MOV DECIMAL2, DL
```

```
; Display result
LEA DX, RESULT_MSG
MOV AH, 09H
INT 21H

MOV AX, AVERAGE
CALL PRINT_NUMBER

LEA DX, DOT_MSG
MOV AH, 09H
INT 21H

MOV DL, DECIMAL1
ADD DL, '0'
MOV AH, 02H
INT 21H

MOV DL, DECIMAL2
ADD DL, '0'
MOV AH, 02H
INT 21H

LEA DX, NEWLINE
MOV AH, 09H
INT 21H

MOV AH, 4CH
INT 21H
MAIN ENDP

READ_NUMBER PROC
    PUSH BX
    PUSH CX
    PUSH DX

    XOR BX, BX

RN_LOOP:
    MOV AH, 01H
    INT 21H

    CMP AL, 0DH
    JE RN_DONE

    CMP AL, '0'
    JB RN_LOOP
    CMP AL, '9'
    JA RN_LOOP

    SUB AL, '0'
    MOV CL, AL
```

```
    MOV AX, BX
    MOV DX, 10
    MUL DX
    ADD AX, CX
    MOV BX, AX
    JMP RN_LOOP

RN_DONE:
    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_NUMBER ENDP

PRINT_NUMBER PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    XOR CX, CX
    MOV BX, 10

    CMP AX, 0
    JNE PN_DIVIDE
    PUSH AX
    INC CX
    JMP PN_PRINT

PN_DIVIDE:
    XOR DX, DX
    DIV BX
    PUSH DX
    INC CX
    CMP AX, 0
    JNE PN_DIVIDE

PN_PRINT:
    POP DX
    ADD DL, '0'
    MOV AH, 02H
    INT 21H
    LOOP PN_PRINT

    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_NUMBER ENDP
```

```
END MAIN
```

3.3 Input

Input

```
How many numbers do you want to enter? 4
Enter number 1: 10
Enter number 2: 20
Enter number 3: 30
Enter number 4: 40
```

3.4 Output

Output

```
The average is: 25.00
```

4 Number Comparison

4.1 Problem Statement

Prompt the user to enter two integers, compare them, and display whether the first is greater, the second is greater, or both are equal.

4.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT1 DB 'Enter first number: $'
    PROMPT2 DB 'Enter second number: $'
    MSG1 DB 0DH, 0AH, 'First number is greater$'
    MSG2 DB 0DH, 0AH, 'Second number is greater$'
    MSG3 DB 0DH, 0AH, 'Both numbers are equal$'
    NUM1 DW ?
    NUM2 DW ?
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
```

```
MOV AX, @DATA
MOV DS, AX

; Prompt for first number
LEA DX, PROMPT1
MOV AH, 09H
INT 21H

CALL READ_NUMBER
MOV NUM1, AX

LEA DX, NEWLINE
MOV AH, 09H
INT 21H

; Prompt for second number
LEA DX, PROMPT2
MOV AH, 09H
INT 21H

CALL READ_NUMBER
MOV NUM2, AX

; Compare numbers
MOV AX, NUM1
CMP AX, NUM2
JG FIRST_GREATER
JL SECOND_GREATER
JE BOTH_EQUAL

FIRST_GREATER:
    LEA DX, MSG1
    JMP DISPLAY_RESULT

SECOND_GREATER:
    LEA DX, MSG2
    JMP DISPLAY_RESULT

BOTH_EQUAL:
    LEA DX, MSG3

DISPLAY_RESULT:
    MOV AH, 09H
    INT 21H

    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    MOV AH, 4CH
    INT 21H
```

```
MAIN ENDP

READ_NUMBER PROC
    PUSH BX
    PUSH CX
    PUSH DX

    XOR BX, BX

RN_LOOP:
    MOV AH, 01H
    INT 21H

    CMP AL, 0DH
    JE RN_DONE

    CMP AL, '0'
    JB RN_LOOP
    CMP AL, '9'
    JA RN_LOOP

    SUB AL, '0'
    MOV CL, AL
    MOV AX, BX
    MOV DX, 10
    MUL DX
    ADD AX, CX
    MOV BX, AX
    JMP RN_LOOP

RN_DONE:
    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_NUMBER ENDP

END MAIN
```

4.3 Input

Input

Enter first number: 25
Enter second number: 15

4.4 Output

Output

First number is greater

5 Number Swapping

5.1 Problem Statement

Swap two numbers input by the user and print the swapped values.

5.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT1 DB 'Enter first number: $'
    PROMPT2 DB 'Enter second number: $'
    BEFORE_MSG DB 0DH, 0AH, 'Before swapping: First = $'
    COMMA_MSG DB ', Second = $'
    AFTER_MSG DB 0DH, 0AH, 'After swapping: First = $'
    NUM1 DW ?
    NUM2 DW ?
    TEMP DW ?
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for first number
    LEA DX, PROMPT1
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV NUM1, AX

    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H
```

```
; Prompt for second number
LEA DX, PROMPT2
MOV AH, 09H
INT 21H

CALL READ_NUMBER
MOV NUM2, AX

; Display before swapping
LEA DX, BEFORE_MSG
MOV AH, 09H
INT 21H

MOV AX, NUM1
CALL PRINT_NUMBER

LEA DX, COMMA_MSG
MOV AH, 09H
INT 21H

MOV AX, NUM2
CALL PRINT_NUMBER

; Perform swap
MOV AX, NUM1
MOV TEMP, AX
MOV AX, NUM2
MOV NUM1, AX
MOV AX, TEMP
MOV NUM2, AX

; Display after swapping
LEA DX, AFTER_MSG
MOV AH, 09H
INT 21H

MOV AX, NUM1
CALL PRINT_NUMBER

LEA DX, COMMA_MSG
MOV AH, 09H
INT 21H

MOV AX, NUM2
CALL PRINT_NUMBER

LEA DX, NEWLINE
MOV AH, 09H
INT 21H

MOV AH, 4CH
```



```
    INT 21H
MAIN  ENDP

READ_NUMBER  PROC
    PUSH BX
    PUSH CX
    PUSH DX

    XOR BX, BX

RN_LOOP:
    MOV AH, 01H
    INT 21H

    CMP AL, 0DH
    JE RN_DONE

    CMP AL, '0'
    JB RN_LOOP
    CMP AL, '9'
    JA RN_LOOP

    SUB AL, '0'
    MOV CL, AL
    MOV AX, BX
    MOV DX, 10
    MUL DX
    ADD AX, CX
    MOV BX, AX
    JMP RN_LOOP

RN_DONE:
    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_NUMBER  ENDP

PRINT_NUMBER  PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    XOR CX, CX
    MOV BX, 10

    CMP AX, 0
    JNE PN_DIVIDE
    PUSH AX
```

```
    INC CX
    JMP PN_PRINT

PN_DIVIDE:
    XOR DX, DX
    DIV BX
    PUSH DX
    INC CX
    CMP AX, 0
    JNE PN_DIVIDE

PN_PRINT:
    POP DX
    ADD DL, '0'
    MOV AH, 02H
    INT 21H
    LOOP PN_PRINT

    POP DX
    POP CX
    POP BX
    POP AX
    RET

PRINT_NUMBER ENDP

END MAIN
```

5.3 Input

Input

Enter first number: 10
Enter second number: 20

5.4 Output

Output

Before swapping: First = 10, Second = 20
After swapping: First = 20, Second = 10

6 Prime Number Checker

6.1 Problem Statement

Prompt user for an integer and determine whether it is prime.

6.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Enter number: $'
    PRIME_MSG DB 0DH, 0AH, 'It is a Prime number$'
    NOT_PRIME_MSG DB 0DH, 0AH, 'It is NOT a Prime number$'
    NUM DW ?
    DIVISOR DW ?
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for number
    LEA DX, PROMPT
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV NUM, AX

    ; Check if num <= 1
    CMP AX, 1
    JBE NOT_PRIME

    ; Check if num == 2
    CMP AX, 2
    JE IS_PRIME

    ; Check if num is even
    TEST AX, 1
    JZ NOT_PRIME

    ; Check odd divisors from 3
    MOV DIVISOR, 3

CHECK_LOOP:
    ; Check if divisor * divisor > num
    MOV AX, DIVISOR
```

```
MUL DIVISOR
CMP AX, NUM
JA IS_PRIME

; Check if num % divisor == 0
MOV AX, NUM
XOR DX, DX
DIV DIVISOR

CMP DX, 0
JE NOT_PRIME

ADD DIVISOR, 2
JMP CHECK_LOOP

IS_PRIME:
    LEA DX, PRIME_MSG
    JMP DISPLAY_RESULT

NOT_PRIME:
    LEA DX, NOT_PRIME_MSG

DISPLAY_RESULT:
    MOV AH, 09H
    INT 21H

    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP

READ_NUMBER PROC
    PUSH BX
    PUSH CX
    PUSH DX

    XOR BX, BX

RN_LOOP:
    MOV AH, 01H
    INT 21H

    CMP AL, 0DH
    JE RN_DONE

    CMP AL, '0'
    JB RN_LOOP
    CMP AL, '9'
```

```
    JA RN_LOOP

    SUB AL, '0'
    MOV CL, AL
    MOV AX, BX
    MOV DX, 10
    MUL DX
    ADD AX, CX
    MOV BX, AX
    JMP RN_LOOP

RN_DONE:
    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_NUMBER ENDP

END MAIN
```

6.3 Input

Input

Enter number: 7

6.4 Output

Output

It is a Prime number

7 Fibonacci Series Generator

7.1 Problem Statement

Prompt for the number of Fibonacci terms, validate input, and print the Fibonacci series up to that many terms.

7.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Enter number of Fibonacci terms to generate: $
    ,
    SPACE DB ' $'
    N DW ?
    FIB1 DW 0
    FIB2 DW 1
    FIB_NEXT DW ?
    COUNTER DW ?
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for number of terms
    LEA DX, PROMPT
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV N, AX

    CMP AX, 0
    JLE EXIT_PROGRAM

    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    MOV COUNTER, 0

    ; Print first term
    MOV AX, COUNTER
    CMP AX, N
    JGE DONE_PRINTING
```

```
MOV AX, FIB1
CALL PRINT_NUMBER
INC COUNTER

; Print second term
MOV AX, COUNTER
CMP AX, N
JGE DONE_PRINTING

LEA DX, SPACE
MOV AH, 09H
INT 21H

MOV AX, FIB2
CALL PRINT_NUMBER
INC COUNTER

FIB_LOOP:
MOV AX, COUNTER
CMP AX, N
JGE DONE_PRINTING

; Calculate next Fibonacci number
MOV AX, FIB1
ADD AX, FIB2
MOV FIB_NEXT, AX

LEA DX, SPACE
MOV AH, 09H
INT 21H

MOV AX, FIB_NEXT
CALL PRINT_NUMBER

; Update fib1 and fib2
MOV AX, FIB2
MOV FIB1, AX
MOV AX, FIB_NEXT
MOV FIB2, AX

INC COUNTER
JMP FIB_LOOP

DONE_PRINTING:
LEA DX, NEWLINE
MOV AH, 09H
INT 21H

EXIT_PROGRAM:
MOV AH, 4CH
INT 21H
```

```
MAIN ENDP

READ_NUMBER PROC
    PUSH BX
    PUSH CX
    PUSH DX

    XOR BX, BX

RN_LOOP:
    MOV AH, 01H
    INT 21H

    CMP AL, 0DH
    JE RN_DONE

    CMP AL, '0'
    JB RN_LOOP
    CMP AL, '9'
    JA RN_LOOP

    SUB AL, '0'
    MOV CL, AL
    MOV AX, BX
    MOV DX, 10
    MUL DX
    ADD AX, CX
    MOV BX, AX
    JMP RN_LOOP

RN_DONE:
    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_NUMBER ENDP

PRINT_NUMBER PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    XOR CX, CX
    MOV BX, 10

    CMP AX, 0
    JNE PN_DIVIDE
    PUSH AX
    INC CX
```



```
        JMP PN_PRINT

PN_DIVIDE:
        XOR DX, DX
        DIV BX
        PUSH DX
        INC CX
        CMP AX, 0
        JNE PN_DIVIDE

PN_PRINT:
        POP DX
        ADD DL, '0'
        MOV AH, 02H
        INT 21H
        LOOP PN_PRINT

        POP DX
        POP CX
        POP BX
        POP AX
        RET

PRINT_NUMBER ENDP

END MAIN
```

7.3 Input

Input

Enter number of Fibonacci terms to generate: 10

7.4 Output

Output

0 1 1 2 3 5 8 13 21 34

8 Factorial Calculator

8.1 Problem Statement

Prompt user for a number and compute its factorial.

8.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Enter number: $'
    RESULT_MSG DB 0DH, 0AH, 'Factorial of $'
    IS_MSG DB ' is: $'
    NUM DW ?
    FACTORIAL DW 1
    COUNTER DW ?
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for number
    LEA DX, PROMPT
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV NUM, AX

    ; Calculate factorial
    CMP AX, 0
    JE ZERO_FACTORIAL

    MOV FACTORIAL, 1
    MOV COUNTER, 1

FACT_LOOP:
    MOV AX, COUNTER
    CMP AX, NUM
    JA DONE_FACTORIAL

    MOV AX, FACTORIAL
    MUL COUNTER
    MOV FACTORIAL, AX

    INC COUNTER
```

```
        JMP FACT_LOOP

ZERO_FACTORIAL:
        MOV FACTORIAL, 1

DONE_FACTORIAL:
        ; Display result message
        LEA DX, RESULT_MSG
        MOV AH, 09H
        INT 21H

        MOV AX, NUM
        CALL PRINT_NUMBER

        LEA DX, IS_MSG
        MOV AH, 09H
        INT 21H

        MOV AX, FACTORIAL
        CALL PRINT_NUMBER

        LEA DX, NEWLINE
        MOV AH, 09H
        INT 21H

        MOV AH, 4CH
        INT 21H
MAIN ENDP

READ_NUMBER PROC
        PUSH BX
        PUSH CX
        PUSH DX
        XOR BX, BX

RN_LOOP:
        MOV AH, 01H
        INT 21H

        CMP AL, 0DH
        JE RN_DONE

        CMP AL, '0'
        JB RN_LOOP
        CMP AL, '9'
        JA RN_LOOP

        SUB AL, '0'
        MOV CL, AL
        MOV AX, BX
        MOV DX, 10
```

```
MUL DX
ADD AX, CX
MOV BX, AX
JMP RN_LOOP

RN_DONE:
MOV AX, BX
POP DX
POP CX
POP BX
RET
READ_NUMBER ENDP

PRINT_NUMBER PROC
PUSH AX
PUSH BX
PUSH CX
PUSH DX

XOR CX, CX
MOV BX, 10

CMP AX, 0
JNE PN_DIVIDE
PUSH AX
INC CX
JMP PN_PRINT

PN_DIVIDE:
XOR DX, DX
DIV BX
PUSH DX
INC CX
CMP AX, 0
JNE PN_DIVIDE

PN_PRINT:
POP DX
ADD DL, '0'
MOV AH, 02H
INT 21H
LOOP PN_PRINT

POP DX
POP CX
POP BX
POP AX
RET
PRINT_NUMBER ENDP

END MAIN
```

8.3 Input

Input

Enter number: 5

8.4 Output

Output

Factorial of 5 is: 120

9 GCD Calculator

9.1 Problem Statement

Compute GCD of two user-input integers using Euclid's algorithm.

9.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT1 DB 'Enter first number: $'
    PROMPT2 DB 'Enter second number: $'
    RESULT_MSG DB 0DH, 0AH, 'GCD is: $'
    NUM1 DW ?
    NUM2 DW ?
    GCD_VAL DW ?
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for first number
    LEA DX, PROMPT1
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV NUM1, AX
```

```
    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    ; Prompt for second number
    LEA DX, PROMPT2
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV NUM2, AX

    ; Calculate GCD using Euclidean algorithm
    MOV AX, NUM1
    MOV BX, NUM2

GCD_LOOP:
    CMP BX, 0
    JE GCD_DONE

    XOR DX, DX
    DIV BX

    MOV AX, BX
    MOV BX, DX
    JMP GCD_LOOP

GCD_DONE:
    MOV GCD_VAL, AX

    ; Display result
    LEA DX, RESULT_MSG
    MOV AH, 09H
    INT 21H

    MOV AX, GCD_VAL
    CALL PRINT_NUMBER

    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP

READ_NUMBER PROC
    PUSH BX
    PUSH CX
    PUSH DX
```

```
        XOR BX, BX

RN_LOOP:
        MOV AH, 01H
        INT 21H

        CMP AL, 0DH
        JE RN_DONE

        CMP AL, '0'
        JB RN_LOOP
        CMP AL, '9'
        JA RN_LOOP

        SUB AL, '0'
        MOV CL, AL
        MOV AX, BX
        MOV DX, 10
        MUL DX
        ADD AX, CX
        MOV BX, AX
        JMP RN_LOOP

RN_DONE:
        MOV AX, BX
        POP DX
        POP CX
        POP BX
        RET

READ_NUMBER ENDP

PRINT_NUMBER PROC
        PUSH AX
        PUSH BX
        PUSH CX
        PUSH DX

        XOR CX, CX
        MOV BX, 10

        CMP AX, 0
        JNE PN_DIVIDE
        PUSH AX
        INC CX
        JMP PN_PRINT

PN_DIVIDE:
        XOR DX, DX
        DIV BX
        PUSH DX
        INC CX
```

```
    CMP AX, 0
    JNE PN_DIVIDE

PN_PRINT:
    POP DX
    ADD DL, '0'
    MOV AH, 02H
    INT 21H
    LOOP PN_PRINT

    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_NUMBER ENDP

END MAIN
```

9.3 Input

Input

```
Enter first number: 12
Enter second number: 8
```

9.4 Output

Output

```
GCD is: 4
```


10 LCM Calculator

10.1 Problem Statement

Compute LCM of two numbers using the formula: $\text{LCM}(a, b) = (a \times b) / \text{GCD}(a, b)$.

10.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT1 DB 'Enter first number: $'
    PROMPT2 DB 'Enter second number: $'
    RESULT_MSG DB 0DH, 0AH, 'LCM is: $'
    NUM1 DW ?
    NUM2 DW ?
    GCD_VAL DW ?
    LCM_VAL DW ?
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for first number
    LEA DX, PROMPT1
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV NUM1, AX

    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    ; Prompt for second number
    LEA DX, PROMPT2
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV NUM2, AX

    ; Calculate GCD
    MOV AX, NUM1
    MOV BX, NUM2
```

```
GCD_LOOP:
    CMP BX, 0
    JE GCD_DONE

    XOR DX, DX
    DIV BX

    MOV AX, BX
    MOV BX, DX
    JMP GCD_LOOP

GCD_DONE:
    MOV GCD_VAL, AX

    ; Calculate LCM = (num1 * num2) / gcd
    MOV AX, NUM1
    MUL NUM2
    DIV GCD_VAL
    MOV LCM_VAL, AX

    ; Display result
    LEA DX, RESULT_MSG
    MOV AH, 09H
    INT 21H

    MOV AX, LCM_VAL
    CALL PRINT_NUMBER

    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP

READ_NUMBER PROC
    PUSH BX
    PUSH CX
    PUSH DX

    XOR BX, BX

RN_LOOP:
    MOV AH, 01H
    INT 21H

    CMP AL, 0DH
    JE RN_DONE

    CMP AL, '0'
```

```
JB RN_LOOP
CMP AL, '9'
JA RN_LOOP

SUB AL, '0'
MOV CL, AL
MOV AX, BX
MOV DX, 10
MUL DX
ADD AX, CX
MOV BX, AX
JMP RN_LOOP

RN_DONE:
MOV AX, BX
POP DX
POP CX
POP BX
RET
READ_NUMBER ENDP

PRINT_NUMBER PROC
PUSH AX
PUSH BX
PUSH CX
PUSH DX

XOR CX, CX
MOV BX, 10

CMP AX, 0
JNE PN_DIVIDE
PUSH AX
INC CX
JMP PN_PRINT

PN_DIVIDE:
XOR DX, DX
DIV BX
PUSH DX
INC CX
CMP AX, 0
JNE PN_DIVIDE

PN_PRINT:
POP DX
ADD DL, '0'
MOV AH, 02H
INT 21H
LOOP PN_PRINT
```

```
    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_NUMBER ENDP

END MAIN
```

10.3 Input

Input

```
Enter first number: 12
Enter second number: 8
```

10.4 Output

Output

```
LCM is: 24
```

11 Left Triangle Pattern

11.1 Problem Statement

Prompt user for height and print left-aligned star triangle of that height.

11.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Enter the height of the triangle: $'
    STAR DB '*$'
    NEWLINE DB 0DH, 0AH, '$'
    HEIGHT DW ?
    ROW DW 1
    COL DW ?

.CODE
MAIN PROC
```

```
MOV AX, @DATA
MOV DS, AX

; Prompt for height
LEA DX, PROMPT
MOV AH, 09H
INT 21H

CALL READ_NUMBER
MOV HEIGHT, AX

LEA DX, NEWLINE
MOV AH, 09H
INT 21H

MOV ROW, 1

ROW_LOOP:
MOV AX, ROW
CMP AX, HEIGHT
JA DONE_TRIANGLE

MOV COL, 1

COL_LOOP:
MOV AX, COL
CMP AX, ROW
JA END_ROW

LEA DX, STAR
MOV AH, 09H
INT 21H
INC COL
JMP COL_LOOP

END_ROW:
LEA DX, NEWLINE
MOV AH, 09H
INT 21H
INC ROW
JMP ROW_LOOP

DONE_TRIANGLE:
MOV AH, 4CH
INT 21H
MAIN ENDP

READ_NUMBER PROC
PUSH BX
PUSH CX
PUSH DX
```

```
    XOR BX, BX

RN_LOOP:
    MOV AH, 01H
    INT 21H
    CMP AL, 0DH
    JE RN_DONE

    CMP AL, '0'
    JB RN_LOOP
    CMP AL, '9'
    JA RN_LOOP
    SUB AL, '0'
    MOV CL, AL
    MOV AX, BX
    MOV DX, 10
    MUL DX
    ADD AX, CX
    MOV BX, AX
    JMP RN_LOOP

RN_DONE:
    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_NUMBER ENDP

END MAIN
```

11.3 Input

Input

Enter the height of the triangle: 6

11.4 Output

Output

```
*
**
***
****
*****
*****
```

12 Star Pyramid Pattern

12.1 Problem Statement

Prompt the user for number of rows and print a centered pyramid of * characters.

12.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Enter number of rows: $'
    STAR DB '*$'
    SPACE DB ' $'
    NEWLINE DB 0DH, 0AH, '$'
    ROWS DW ?
    CURRENT_ROW DW 1
    SPACES DW ?
    STARS DW ?
    COUNTER DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for rows
    LEA DX, PROMPT
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV ROWS, AX

    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    MOV CURRENT_ROW, 1

ROW_LOOP:
    MOV AX, CURRENT_ROW
    CMP AX, ROWS
    JA DONE_PYRAMID

    ; Calculate spaces = rows - currentRow
    MOV AX, ROWS
    SUB AX, CURRENT_ROW
    MOV SPACES, AX
```

```
    ; Calculate stars = 2 * currentRow - 1
    MOV AX, CURRENT_ROW
    MOV BX, 2
    MUL BX
    DEC AX
    MOV STARS, AX

    ; Print spaces
    MOV COUNTER, 0
SPACE_LOOP:
    MOV AX, COUNTER
    CMP AX, SPACES
    JAE PRINT_STARS

    LEA DX, SPACE
    MOV AH, 09H
    INT 21H

    INC COUNTER
    JMP SPACE_LOOP

PRINT_STARS:
    MOV COUNTER, 0
STAR_LOOP:
    MOV AX, COUNTER
    CMP AX, STARS
    JAE END_ROW

    LEA DX, STAR
    MOV AH, 09H
    INT 21H

    INC COUNTER
    JMP STAR_LOOP

END_ROW:
    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    INC CURRENT_ROW
    JMP ROW_LOOP

DONE_PYRAMID:
    MOV AH, 4CH
    INT 21H
MAIN ENDP

READ_NUMBER PROC
    PUSH BX
```



```
PUSH CX
PUSH DX
XOR BX, BX

RN_LOOP:
    MOV AH, 01H
    INT 21H
    CMP AL, 0DH
    JE RN_DONE
    CMP AL, '0'
    JB RN_LOOP
    CMP AL, '9'
    JA RN_LOOP
    SUB AL, '0'
    MOV CL, AL
    MOV AX, BX
    MOV DX, 10
    MUL DX
    ADD AX, CX
    MOV BX, AX
    JMP RN_LOOP

RN_DONE:
    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_NUMBER ENDP

END MAIN
```

12.3 Input

Input

Enter number of rows: 5

12.4 Output

Output

```
  *
 ***
*****
*****
*****
```

13 Matrix Addition

13.1 Problem Statement

Input two matrices (user enters rows, cols, then elements) and display their sum ($A + B$).

13.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT_ROWS DB 'Enter number of rows: $'
    PROMPT_COLS DB 'Enter number of columns: $'
    PROMPT_MAT1 DB 0DH, 0AH, 'Enter elements of Matrix 1:', 0
                DH, 0AH, '$'
    PROMPT_MAT2 DB 0DH, 0AH, 'Enter elements of Matrix 2:', 0
                DH, 0AH, '$'
    PROMPT_ELEMENT DB 'Enter element [$'
    CLOSE_BRACKET DB '][$'
    COLON_MSG DB ']: $'
    RESULT_HEADER DB 0DH, 0AH, 'Resultant Matrix (A + B):', 0
                DH, 0AH, '$'
    SPACE DB ' $'
    NEWLINE DB 0DH, 0AH, '$'

    ROWS DW ?
    COLS DW ?
    MATRIX1 DW 100 DUP(?)
    MATRIX2 DW 100 DUP(?)
    RESULT DW 100 DUP(?)

    CURRENT_ROW DW ?
    CURRENT_COL DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for rows
    LEA DX, PROMPT_ROWS
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER
    MOV ROWS, AX

    LEA DX, NEWLINE
```

```
MOV AH, 09H
INT 21H

; Prompt for columns
LEA DX, PROMPT_COLS
MOV AH, 09H
INT 21H
CALL READ_NUMBER
MOV COLS, AX

; Read Matrix 1
LEA DX, PROMPT_MAT1
MOV AH, 09H
INT 21H
MOV CURRENT_ROW, 0
MAT1_ROW_LOOP:
MOV AX, CURRENT_ROW
CMP AX, ROWS
JAE MAT1_DONE
MOV CURRENT_COL, 0
MAT1_COL_LOOP:
MOV AX, CURRENT_COL
CMP AX, COLS
JAE MAT1_NEXT_ROW

LEA DX, PROMPT_ELEMENT
MOV AH, 09H
INT 21H
MOV AX, CURRENT_ROW
CALL PRINT_NUMBER

LEA DX, CLOSE_BRACKET
MOV AH, 09H
INT 21H
MOV AX, CURRENT_COL
CALL PRINT_NUMBER
LEA DX, COLON_MSG
MOV AH, 09H
INT 21H

CALL READ_NUMBER

PUSH AX
MOV AX, CURRENT_ROW
MUL COLS
ADD AX, CURRENT_COL
MOV BX, 2
MUL BX
MOV SI, AX
POP AX
MOV MATRIX1[SI], AX
```

```
    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    INC CURRENT_COL
    JMP MAT1_COL_LOOP

MAT1_NEXT_ROW:
    INC CURRENT_ROW
    JMP MAT1_ROW_LOOP

MAT1_DONE:
    ; Read Matrix 2
    LEA DX, PROMPT_MAT2
    MOV AH, 09H
    INT 21H

    MOV CURRENT_ROW, 0
MAT2_ROW_LOOP:
    MOV AX, CURRENT_ROW
    CMP AX, ROWS
    JAE MAT2_DONE

    MOV CURRENT_COL, 0
MAT2_COL_LOOP:
    MOV AX, CURRENT_COL
    CMP AX, COLS
    JAE MAT2_NEXT_ROW

    LEA DX, PROMPT_ELEMENT
    MOV AH, 09H
    INT 21H

    MOV AX, CURRENT_ROW
    CALL PRINT_NUMBER

    LEA DX, CLOSE_BRACKET
    MOV AH, 09H
    INT 21H

    MOV AX, CURRENT_COL
    CALL PRINT_NUMBER

    LEA DX, COLON_MSG
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER

    PUSH AX
```

```
MOV AX, CURRENT_ROW
MUL COLS
ADD AX, CURRENT_COL
MOV BX, 2
MUL BX
MOV SI, AX
POP AX
MOV MATRIX2[SI], AX

LEA DX, NEWLINE
MOV AH, 09H
INT 21H

INC CURRENT_COL
JMP MAT2_COL_LOOP

MAT2_NEXT_ROW:
INC CURRENT_ROW
JMP MAT2_ROW_LOOP

MAT2_DONE:
; Perform addition
MOV CURRENT_ROW, 0
ADD_ROW_LOOP:
MOV AX, CURRENT_ROW
CMP AX, ROWS
JAE ADD_DONE

MOV CURRENT_COL, 0
ADD_COL_LOOP:
MOV AX, CURRENT_COL
CMP AX, COLS
JAE ADD_NEXT_ROW

MOV AX, CURRENT_ROW
MUL COLS
ADD AX, CURRENT_COL
MOV BX, 2
MUL BX
MOV SI, AX

MOV AX, MATRIX1[SI]
ADD AX, MATRIX2[SI]
MOV RESULT[SI], AX

INC CURRENT_COL
JMP ADD_COL_LOOP

ADD_NEXT_ROW:
INC CURRENT_ROW
JMP ADD_ROW_LOOP
```

```
ADD_DONE:
    ; Display result header
    LEA DX, RESULT_HEADER
    MOV AH, 09H
    INT 21H

    ; Print result matrix
    MOV CURRENT_ROW, 0
PRINT_ROW_LOOP:
    MOV AX, CURRENT_ROW
    CMP AX, ROWS
    JAE PRINT_DONE

    MOV CURRENT_COL, 0
PRINT_COL_LOOP:
    MOV AX, CURRENT_COL
    CMP AX, COLS
    JAE PRINT_NEXT_ROW

    MOV AX, CURRENT_ROW
    MUL COLS
    ADD AX, CURRENT_COL
    MOV BX, 2
    MUL BX
    MOV SI, AX

    MOV AX, RESULT[SI]
    CALL PRINT_NUMBER

    MOV AX, CURRENT_COL
    INC AX
    CMP AX, COLS
    JAE SKIP_SPACE

    LEA DX, SPACE
    MOV AH, 09H
    INT 21H

SKIP_SPACE:
    INC CURRENT_COL
    JMP PRINT_COL_LOOP

PRINT_NEXT_ROW:
    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    INC CURRENT_ROW
    JMP PRINT_ROW_LOOP
```

```
PRINT_DONE:
    MOV AH, 4CH
    INT 21H
MAIN ENDP

READ_NUMBER PROC
    PUSH BX
    PUSH CX
    PUSH DX
    XOR BX, BX

RN_LOOP:
    MOV AH, 01H
    INT 21H
    CMP AL, 0DH
    JE RN_DONE
    CMP AL, '0'
    JB RN_LOOP
    CMP AL, '9'
    JA RN_LOOP

    SUB AL, '0'
    MOV CL, AL
    MOV AX, BX
    MOV DX, 10
    MUL DX
    ADD AX, CX
    MOV BX, AX
    JMP RN_LOOP

RN_DONE:
    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_NUMBER ENDP

PRINT_NUMBER PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
    XOR CX, CX
    MOV BX, 10
    CMP AX, 0
    JNE PN_DIVIDE
    PUSH AX
    INC CX
    JMP PN_PRINT
```

```
PN_DIVIDE:
    XOR DX, DX
    DIV BX
    PUSH DX
    INC CX
    CMP AX, 0
    JNE PN_DIVIDE

PN_PRINT:
    POP DX
    ADD DL, '0'
    MOV AH, 02H
    INT 21H
    LOOP PN_PRINT
    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_NUMBER ENDP

END MAIN
```

13.3 Input

Input

```
Enter number of rows: 2
Enter number of columns: 2
Enter elements of Matrix 1:
Enter element [0][0]: 1
Enter element [0][1]: 2
Enter element [1][0]: 3
Enter element [1][1]: 4
Enter elements of Matrix 2:
Enter element [0][0]: 5
Enter element [0][1]: 6
Enter element [1][0]: 7
Enter element [1][1]: 8
```

13.4 Output

Output

```
Resultant Matrix (A + B):
6 8
10 12
```


14 Matrix Subtraction

14.1 Problem Statement

Input two matrices(user enters rows, cols, then elements) and display their subtraction ($A - B$).

14.2 Assembly Code

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT_ROWS DB 'Enter number of rows: $'
    PROMPT_COLS DB 'Enter number of columns: $'
    PROMPT_MAT1 DB 0DH, 0AH, 'Enter elements of Matrix 1:', 0
                DH, 0AH, '$'
    PROMPT_MAT2 DB 0DH, 0AH, 'Enter elements of Matrix 2:', 0
                DH, 0AH, '$'
    PROMPT_ELEMENT DB 'Enter element [$'
    CLOSE_BRACKET DB '][$'
    COLON_MSG DB ']: $'
    RESULT_HEADER DB 0DH, 0AH, 'Resultant Matrix (A - B):', 0
                DH, 0AH, '$'
    SPACE DB ' $'
    NEWLINE DB 0DH, 0AH, '$'
    ROWS DW ?
    COLS DW ?
    MATRIX1 DW 100 DUP(?)
    MATRIX2 DW 100 DUP(?)
    RESULT DW 100 DUP(?)

    CURRENT_ROW DW ?
    CURRENT_COL DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for rows
    LEA DX, PROMPT_ROWS
    MOV AH, 09H
    INT 21H
    CALL READ_NUMBER
    MOV ROWS, AX
    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H
```

```
; Prompt for columns
LEA DX, PROMPT_COLS
MOV AH, 09H
INT 21H
CALL READ_NUMBER
MOV COLS, AX

; Read Matrix 1
LEA DX, PROMPT_MAT1
MOV AH, 09H
INT 21H

MOV CURRENT_ROW, 0

MAT1_ROW_LOOP:
MOV AX, CURRENT_ROW
CMP AX, ROWS
JAE MAT1_DONE
MOV CURRENT_COL, 0

MAT1_COL_LOOP:
MOV AX, CURRENT_COL
CMP AX, COLS
JAE MAT1_NEXT_ROW
LEA DX, PROMPT_ELEMENT
MOV AH, 09H
INT 21H
MOV AX, CURRENT_ROW
CALL PRINT_NUMBER
LEA DX, CLOSE_BRACKET
MOV AH, 09H
INT 21H
MOV AX, CURRENT_COL
CALL PRINT_NUMBER
LEA DX, COLON_MSG
MOV AH, 09H
INT 21H

CALL READ_NUMBER

PUSH AX
MOV AX, CURRENT_ROW
MUL COLS
ADD AX, CURRENT_COL
MOV BX, 2
MUL BX
MOV SI, AX
POP AX
MOV MATRIX1[SI], AX
LEA DX, NEWLINE
MOV AH, 09H
```

```
    INT 21H
    INC CURRENT_COL
    JMP MAT1_COL_LOOP

MAT1_NEXT_ROW:
    INC CURRENT_ROW
    JMP MAT1_ROW_LOOP

MAT1_DONE:
    ; Read Matrix 2
    LEA DX, PROMPT_MAT2
    MOV AH, 09H
    INT 21H
    MOV CURRENT_ROW, 0

MAT2_ROW_LOOP:
    MOV AX, CURRENT_ROW
    CMP AX, ROWS
    JAE MAT2_DONE
    MOV CURRENT_COL, 0

MAT2_COL_LOOP:
    MOV AX, CURRENT_COL
    CMP AX, COLS
    JAE MAT2_NEXT_ROW
    LEA DX, PROMPT_ELEMENT
    MOV AH, 09H
    INT 21H
    MOV AX, CURRENT_ROW
    CALL PRINT_NUMBER
    LEA DX, CLOSE_BRACKET
    MOV AH, 09H
    INT 21H
    MOV AX, CURRENT_COL
    CALL PRINT_NUMBER
    LEA DX, COLON_MSG
    MOV AH, 09H
    INT 21H

    CALL READ_NUMBER

    PUSH AX
    MOV AX, CURRENT_ROW
    MUL COLS
    ADD AX, CURRENT_COL
    MOV BX, 2
    MUL BX
    MOV SI, AX
    POP AX
    MOV MATRIX2[SI], AX
```

```
    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H
    INC CURRENT_COL
    JMP MAT2_COL_LOOP

MAT2_NEXT_ROW:
    INC CURRENT_ROW
    JMP MAT2_ROW_LOOP

MAT2_DONE:
    ; Perform subtraction
    MOV CURRENT_ROW, 0

SUB_ROW_LOOP:
    MOV AX, CURRENT_ROW
    CMP AX, ROWS
    JAE SUB_DONE

    MOV CURRENT_COL, 0
SUB_COL_LOOP:
    MOV AX, CURRENT_COL
    CMP AX, COLS
    JAE SUB_NEXT_ROW
    MOV AX, CURRENT_ROW
    MUL COLS
    ADD AX, CURRENT_COL
    MOV BX, 2
    MUL BX
    MOV SI, AX
    MOV AX, MATRIX1[SI]
    SUB AX, MATRIX2[SI]
    MOV RESULT[SI], AX
    INC CURRENT_COL
    JMP SUB_COL_LOOP

SUB_NEXT_ROW:
    INC CURRENT_ROW
    JMP SUB_ROW_LOOP

SUB_DONE:
    ; Display result header
    LEA DX, RESULT_HEADER
    MOV AH, 09H
    INT 21H

    ; Print result matrix
    MOV CURRENT_ROW, 0
PRINT_ROW_LOOP:
    MOV AX, CURRENT_ROW
    CMP AX, ROWS
```

```
JAE PRINT_DONE

MOV CURRENT_COL, 0
PRINT_COL_LOOP:
MOV AX, CURRENT_COL
CMP AX, COLS
JAE PRINT_NEXT_ROW

MOV AX, CURRENT_ROW
MUL COLS
ADD AX, CURRENT_COL
MOV BX, 2
MUL BX
MOV SI, AX
MOV AX, RESULT[SI]
CALL PRINT_NUMBER
MOV AX, CURRENT_COL
INC AX
CMP AX, COLS
JAE SKIP_SPACE
LEA DX, SPACE
MOV AH, 09H
INT 21H

SKIP_SPACE:
INC CURRENT_COL
JMP PRINT_COL_LOOP

PRINT_NEXT_ROW:
LEA DX, NEWLINE
MOV AH, 09H
INT 21H

INC CURRENT_ROW
JMP PRINT_ROW_LOOP

PRINT_DONE:
MOV AH, 4CH
INT 21H
MAIN ENDP

READ_NUMBER PROC
PUSH BX
PUSH CX
PUSH DX

XOR BX, BX

RN_LOOP:
MOV AH, 01H
INT 21H
```

```
CMP AL, 0DH
JE RN_DONE

CMP AL, '0'
JB RN_LOOP
CMP AL, '9'
JA RN_LOOP

SUB AL, '0'
MOV CL, AL
MOV AX, BX
MOV DX, 10
MUL DX
ADD AX, CX
MOV BX, AX
JMP RN_LOOP

RN_DONE:
MOV AX, BX
POP DX
POP CX
POP BX
RET

READ_NUMBER ENDP

PRINT_NUMBER PROC
PUSH AX
PUSH BX
PUSH CX
PUSH DX
XOR CX, CX
MOV BX, 10
CMP AX, 0
JNE PN_DIVIDE
PUSH AX
INC CX
JMP PN_PRINT

PN_DIVIDE:
XOR DX, DX
DIV BX
PUSH DX
INC CX
CMP AX, 0
JNE PN_DIVIDE

PN_PRINT:
POP DX
ADD DL, '0'
MOV AH, 02H
```

```
    INT 21H
    LOOP PN_PRINT

    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_NUMBER ENDP

END MAIN
```

14.3 Input

Input

```
Enter number of rows:  2
Enter number of columns: 2
Enter elements of Matrix 1:
Enter element [0][0]: 10
Enter element [0][1]: 12
Enter element [1][0]:  8
Enter element [1][1]: 15
Enter elements of Matrix 2:
Enter element [0][0]:  5
Enter element [0][1]:  6
Enter element [1][0]:  3
Enter element [1][1]:  7
```

14.4 Output

Output

```
Resultant Matrix (A - B):
5 6
5 8
```

End of Report

*Prepared by: Debashish Chakraborty
Date: November 5, 2025*